

**МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
КАФЕДРА ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ**

ОТЧЁТ
по лабораторной работе №2
по дисциплине «Основы машинного обучения»
Тема: ПРИМЕНЕНИЕ МЕТОДОВ МАШИННОГО ОБУЧЕНИЯ И
ИХ ОЦЕНКА ДЛЯ ЗАДАЧ КЛАССИФИКАЦИИ

Студент гр. 3312	_____	Барченков П.А.
Студент гр. 3312	_____	Лебедев И.А.
Студент гр. 3312	_____	Шарапов И.Д.
Преподаватель	_____	Петруша П.Г.

Санкт-Петербург
2025

Содержание

ЦЕЛЬ РАБОТЫ	3
ОПИСАНИЕ ЗАДАНИЯ	3
1. Тестирование моделей	3
2. Оценка ROC-AUC	5
3. Значимость признаков для Random Forest.....	6
ВЫВОДЫ	7
ПРИЛОЖЕНИЕ	8

ЦЕЛЬ РАБОТЫ

Получение и закрепление навыков в применении методов машинного обучения и их оценки для задач классификации.

ОПИСАНИЕ ЗАДАНИЯ

Перед нами стоит задачи классификации уровня спроса demand. Для этого в самом начале уникальным строковым значениям demand сопоставили числа. Выписали в массив features остальные признаки товаров. Разделили данные на тренировочные и тестовые. После этого произвели последовательное тестирование каждой модели.

1. Тестирование моделей

- `KNeighborsClassifier(n_neighbors=15)`
Accuracy: 0.8701 Precision: 0.8305 Recall: 0.8701 F1-Score: 0.8316
- `SVC(kernel='linear', C=10, random_state=42)`
Accuracy: 0.8651 Precision: 0.8833 Recall: 0.8651 F1-Score: 0.8025
- `RandomForestClassifier(n_estimators=100, random_state=42)`
Accuracy: 0.8575 Precision: 0.8290 Recall: 0.8575 F1-Score: 0.8383
- `DecisionTreeClassifier(random_state=42)`
Accuracy: 0.8186 Precision: 0.8185 Recall: 0.8186 F1-Score: 0.8185
- `LogisticRegression(random_state=42, max_iter=1000)`
Accuracy: 0.8643 Precision: 0.7571 Recall: 0.8643 F1-Score: 0.8023

Также произвели поиск оптимальных параметров для Random Forest с помощью Grid Search:

Optimized Random Forest: Accuracy: 0.8705 Precision: 0.8332 Recall: 0.8705 F1-Score: 0.8294

Best params: {'max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 100}

Best score: 0.8746

Построили гистограмму (рис. 1), чтобы сравнить модели:

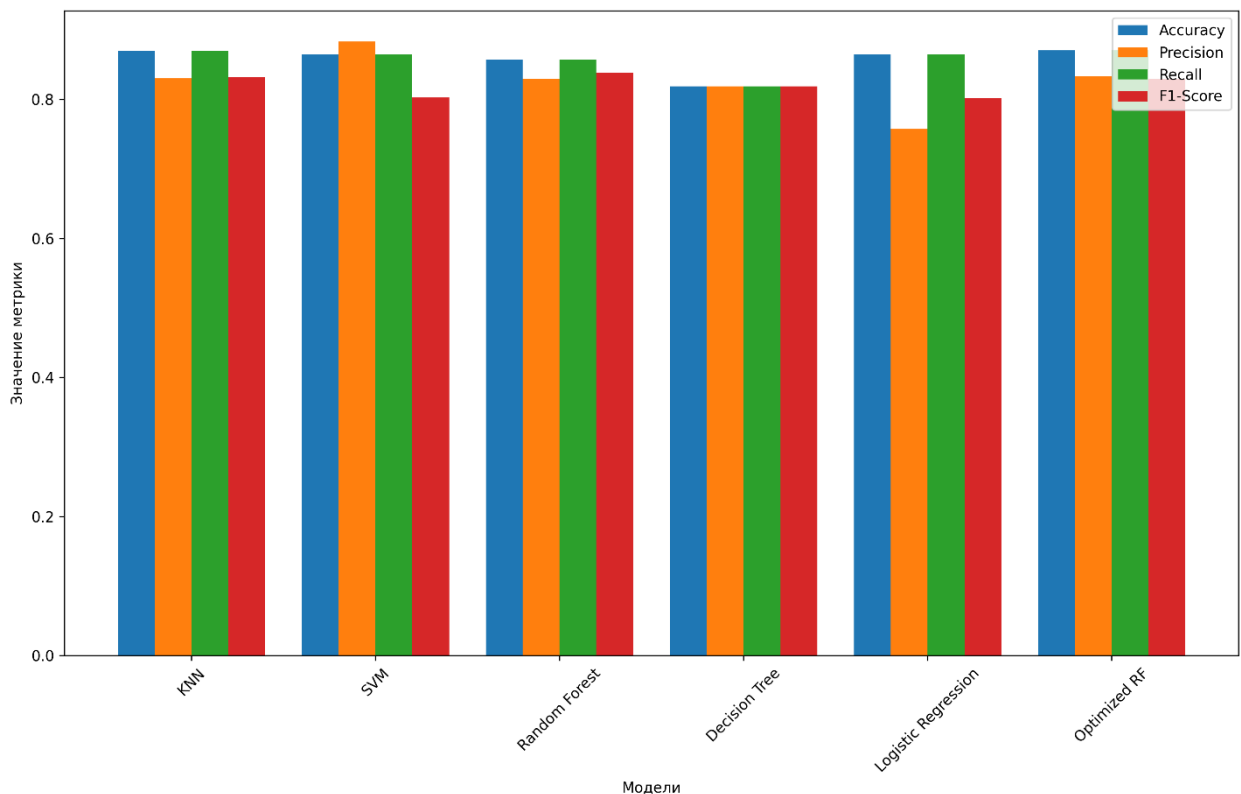


Рисунок 1 – Гистограмма оценок моделей

На рисунке видно, что:

- Лучший показатель Accuracy достигается при «оптимизированном» Random Forest (0.8705), незначительно превосходя KNN (0.8701) и Logistic Regression (0.8643). Наименьший показатель у Decision Tree (0.8186).
- Лучший показатель Precision у SVM (0.8833), за ним следует Optimized RF (0.8332) и KNN (0.8305). Logistic Regression значительно отстаёт (0.7571).
- Лучший показатель Recall достигается при «оптимизированном» Random Forest (0.8705), незначительно превосходя KNN (0.8701). Decision Tree показывает 0.8186.
- Лучший показатель F1-Score у Random Forest (0.8383), за ним следует KNN (0.8316). Decision Tree показывает 0.8185.

2. Оценка ROC-AUC

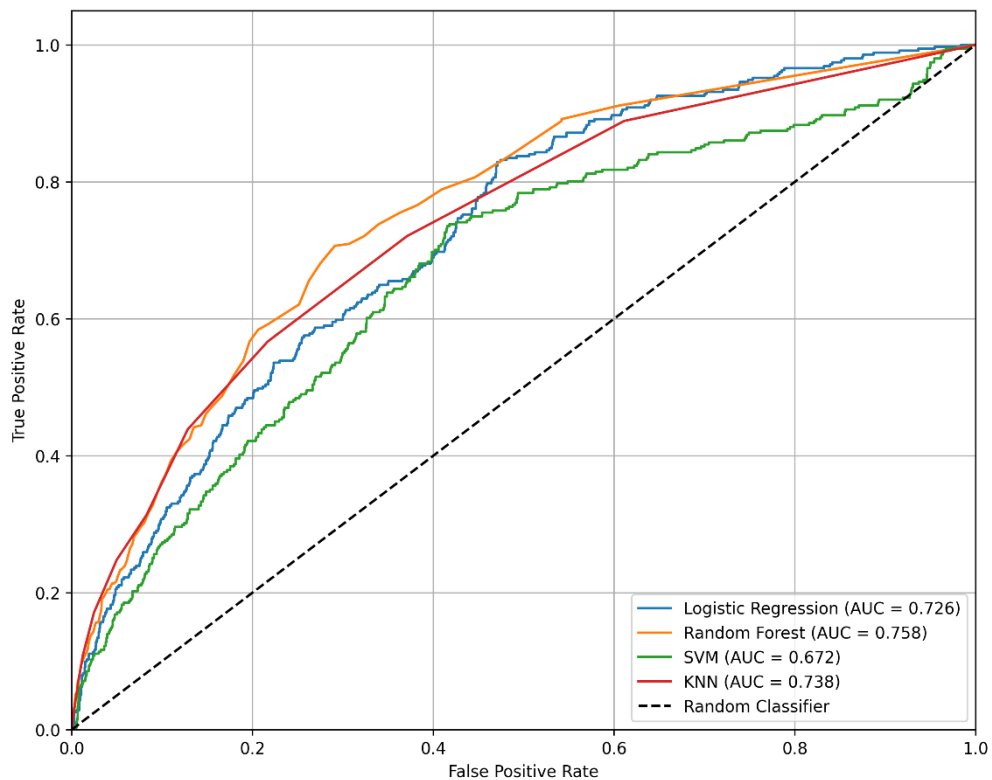


Рисунок 2 – Кривые ROC-AUC

1. Random Forest (AUC = 0.758) показывает наилучшую способность различать товары с высоким и низким спросом при изменении порога классификации. Его ROC-кривая наиболее удалена от диагонали случайного классификатора, что свидетельствует о балансе между чувствительностью и специфичностью.

2. KNN (AUC = 0.738) демонстрирует вторую по величине площадь под ROC-кривой, что говорит о хорошей способности к ранжированию положительных примеров.

3. Logistic Regression (AUC = 0.726) имеет умеренный уровень AUC. Это означает, что линейная модель корректно разделяет классы, но при этом слабее справляется со сложными нелинейными паттернами в данных.

4. SVM (AUC = 0.672) показывает наименьшую площадь под ROC-кривой. Модель хуже различает классы при всех возможных порогах.

3. Значимость признаков для Random Forest

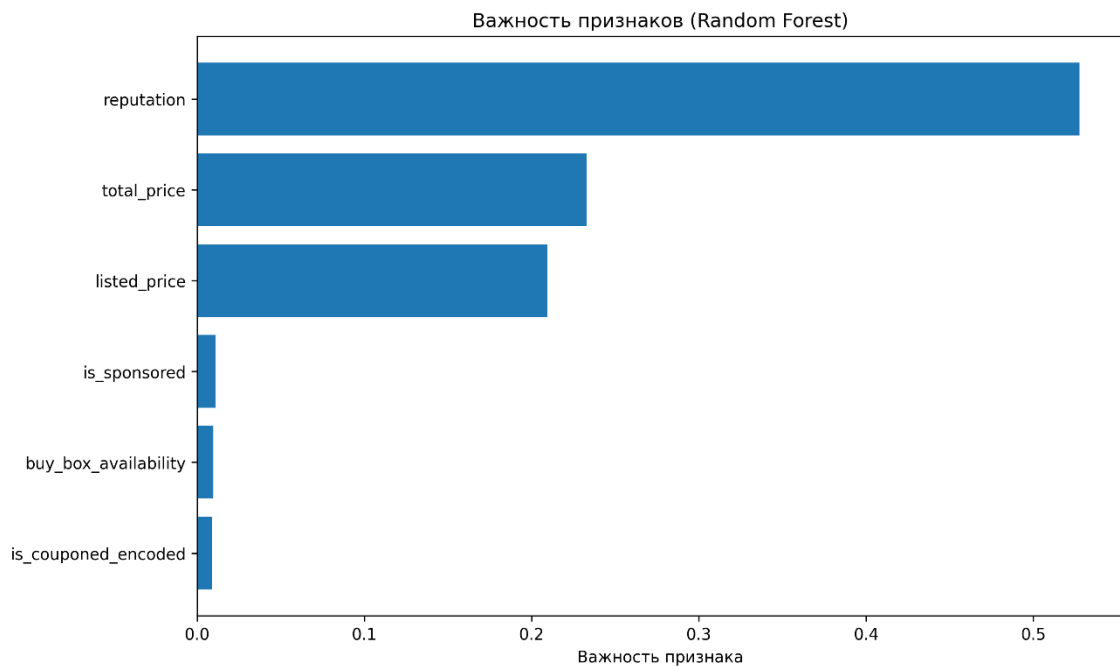


Рисунок 3 – Важность признаков для Random Forest

На основе построенной модели Random Forest наиболее значимыми признаками для предсказания уровня спроса оказались:

- Reputation. Самая высокая важность (~ 0.54) указывает, что рейтинг продавца является ключевым фактором при прогнозировании спроса.
- Total_price. Второй по важности признак (~ 0.22) демонстрирует, что итоговая цена с учётом скидок существенно влияет на популярность товара.
- Listed_price. Текущая цена без учёта купонов и акций (~ 0.20) также играет заметную роль, но чуть меньше, чем итоговая.
- Is_sponsored, Buy_box_availability и Is_couponed_encoded. Эти бинарные индикаторы имеют пренебрежимо малый вклад (< 0.02).

ВЫВОДЫ

В ходе лабораторной работы были выполнены оценки моделей классификации покупок на Amazon по признаку demand.

Среди протестированных алгоритмов наилучшие результаты показал «оптимизированный» Random Forest: он продемонстрировал самую высокую точность (0.8705), полноту (0.8705) и сбалансированное значение F1-Score (0.8294). Именно эта модель обеспечивает наилучший компромисс между чувствительностью и специфичностью и пригодна для использования в прикладных системах прогнозирования спроса.

- SVM продемонстрировал максимум precision (0.8833), что делает его эффективным при необходимости минимизировать ложноположительные ошибки.
- KNN показал стабильные средние показатели по всем метрикам, но уступает Random Forest в целом.
- Logistic Regression обладает хорошей полнотой (0.8643), но страдает от сравнительно низкой точности (0.7571), что снижает её F1-Score.
- Decision Tree продемонстрировал наихудшие результаты.

Random Forest имеет наибольшую площадь под ROC-кривой (0.758), что подтверждает его способность корректно ранжировать примеры при разных порогах. KNN (0.738) и Logistic Regression (0.726) занимают средние позиции, а SVM (0.672) – самое низкое значение.

Важность признаков:

- Reputation (0.54) оказался главным предиктором спроса – рейтинг продавца определяет доверие покупателей.
- Total_price (0.22) и listed_price (0.20) существенно влияют на решение о покупке.
- Is_sponsored, buy_box_availability, is_couponed_encoded в текущей задаче оказались статистически малоинформативными (<0.02 каждого).

ПРИЛОЖЕНИЕ

Код программы

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
roc_auc_score, roc_curve
from sklearn.preprocessing import LabelEncoder

data = pd.read_csv('data/amazon products sales data cleaned.csv')

print(f"Data size: {data.shape}")
print(data.head().to_string())

# Преобразуем строковые значения demand в уникальные метки
label_encoder = LabelEncoder()
data['is_couponed_encoded'] = label_encoder.fit_transform(data['is_couponed'])
y = label_encoder.fit_transform(data['demand'])

features = ['reputation', 'total price', 'listed price', 'is sponsored',
            'is_couponed_encoded', 'buy box availability']
X = data[features]

# Деление на train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42,
                                                    stratify=y)

# Функция оценки модели
def evaluate_model(name, model, X_test, y_test):
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, average='weighted', zero_division=1)
    recall = recall_score(y_test, y_pred, average='weighted', zero_division=1)
    f1 = f1_score(y_test, y_pred, average='weighted', zero_division=1)
    print(f"\n{name}: Accuracy: {accuracy:.4f} Precision: {precision:.4f} Recall:
{recall:.4f} F1-Score: {f1:.4f}")
    return accuracy, precision, recall, f1

results = {}

# KNN
knn = KNeighborsClassifier(n_neighbors=15)
knn.fit(X_train, y_train)
results['KNN'] = evaluate_model('KNN', knn, X_test, y_test)
'''KNN: Accuracy: 0.8701 Precision: 0.8305 Recall: 0.8701 F1-Score: 0.8316'''

# SVM
svm = SVC(kernel='linear', C=10, random_state=42)
svm.fit(X_train, y_train)
results['SVM'] = evaluate_model('SVM', svm, X_test, y_test)
'''SVM: Accuracy: 0.8651 Precision: 0.8833 Recall: 0.8651 F1-Score: 0.8025'''

# Random Forest
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
results['Random Forest'] = evaluate_model('Random Forest', rf, X_test, y_test)
'''Random Forest: Accuracy: 0.8575 Precision: 0.8290 Recall: 0.8575 F1-Score: 0.8383'''

# Decision Tree
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)
results['Decision Tree'] = evaluate_model('Decision Tree', dt, X_test, y_test)
'''Decision Tree: Accuracy: 0.8186 Precision: 0.8185 Recall: 0.8186 F1-Score: 0.8185'''

# Logistic Regression
lr = LogisticRegression(random_state=42, max_iter=1000)
lr.fit(X_train, y_train)
```



```

results['Logistic Regression'] = evaluate_model('Logistic Regression', lr, X test, y test)
'''Logistic Regression: Accuracy: 0.8643 Precision: 0.7571 Recall: 0.8643 F1-Score:
0.8023'''

# Grid Search
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}
rf_grid = RandomForestClassifier(random_state=42)
grid_search = GridSearchCV(rf_grid, param_grid, cv=5, scoring='accuracy', n_jobs=-1)
grid_search.fit(X_train, y_train)
best_rf = grid_search.best_estimator_
results['Optimized RF'] = evaluate_model('Optimized Random Forest', best_rf, X_test, y_test)
print(f"Best params: {grid_search.best_params}")
print(f"Best score: {grid_search.best_score:.4f}")
'''Optimized Random Forest: Accuracy: 0.8705 Precision: 0.8332 Recall: 0.8705 F1-Score:
0.8294
Best params: {'max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 2,
'n_estimators': 100}
Best score: 0.8746'''

# Гистограмма оценок моделей
results_df = pd.DataFrame(results, index=['Accuracy', 'Precision', 'Recall', 'F1-Score']).T
plt.figure(figsize=(12, 8))
metrics = ['Accuracy', 'Precision', 'Recall', 'F1-Score']
x = np.arange(len(results.keys()))
width = 0.2
for i, metric in enumerate(metrics):
    values = [results[model][i] for model in results.keys()]
    plt.bar(x + i * width, values, width, label=metric)
plt.xlabel('Модели')
plt.ylabel('Значение метрики')
plt.title('Сравнение моделей классификации')
plt.xticks(x + width * 1.5, results.keys(), rotation=45)
plt.legend()
plt.tight_layout()
plt.savefig('images/models_comparison.png', dpi=300, bbox_inches='tight')
plt.close()

# ROC-AUC анализ для high demand
y_binary = (y == 2).astype(int)
y_train_binary = (y_train == 2).astype(int)
y_test_binary = (y_test == 2).astype(int)
plt.figure(figsize=(10, 8))
models_binary = {
    'Logistic Regression': LogisticRegression(random_state=42, max_iter=1000),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42),
    'SVM': SVC(kernel='linear', C=10, random_state=42),
    'KNN': KNeighborsClassifier(n_neighbors=15)
}
for name, model in models_binary.items():
    model.fit(X_train, y_train_binary)
    if hasattr(model, 'predict_proba'):
        y_pred_proba = model.predict_proba(X_test)[:, 1]
    else:
        y_pred_proba = model.decision_function(X_test)
    fpr, tpr, _ = roc_curve(y_test_binary, y_pred_proba)
    auc_score = roc_auc_score(y_test_binary, y_pred_proba)
    plt.plot(fpr, tpr, label=f'{name} (AUC = {auc_score:.3f})')
plt.plot([0, 1], [0, 1], 'k--', label='Random Classifier')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC-кривые для предсказания высокого спроса')
plt.legend(loc="lower right")
plt.grid(True)
plt.savefig('images/roc_curves.png', dpi=300, bbox_inches='tight')
plt.close()

# Feature Importance для Random Forest
feature_importance = rf.feature_importances_
feature_names = features
importance_df = pd.DataFrame({
    'feature': feature_names,
    'importance': feature_importance
})

```

```

}).sort values('importance', ascending=False)
plt.figure(figsize=(10, 6))
plt.barh(range(len(importance df)), importance df['importance'])
plt.yticks(range(len(importance df)), importance df['feature'])
plt.xlabel('Важность признака')
plt.title('Важность признаков (Random Forest)')
plt.gca().invert yaxis()
plt.tight layout()
plt.savefig('images/feature importance.png', dpi=300, bbox inches='tight')
plt.close()

```

Ссылка на GitHub: https://github.com/DexTver/ML_LETI/tree/lab_02